

Vibe Coding 的關鍵思維 先當架構師

在 AI 協作開發的時代，你的價值不再是「寫了多少行程式碼」，而是「做出了多少關鍵決策」。

這是一場從鍵盤到思維的革命，而你需要的第一個身份，不是工程師，是架構師。



工程決策權

從鍵盤移到腦袋裡

Vibe coding 的本質不是「少寫 code」，而是把「工程決策權」從鍵盤移到腦袋裡。

真正會翻車的開發者，幾乎都有同一個問題：**一拿到工具就開始做，一開始做就不知道為什麼要做，最後卡住只剩下問「為什麼壞了」。**

這不是工程問題，是架構思維缺失。



傳統工程師 vs 架構師思維

1 問題起手式

傳統：我要怎麼寫？

架構師：這個系統要解什麼問題？

3 面對 AI

傳統：AI 幫我寫 code

架構師：AI 是我下面的執行單位

2 工具使用

傳統：框架、語言導向

架構師：能力模組導向

4 出錯時

傳統：Debug code

架構師：Debug decision

💡 在 vibe coding 時代，成敗關鍵不在於技術熟不熟，而是切得對不對。
架構師思維才是 vibe coding 的必備技能。

Vibe coding 不是給 想學寫程式的人 是給想設計系統的人

如果你還在想「我要怎麼學更多程式語法」，那你可能走錯了方向。

Vibe coding 需要的是：

- 系統思維
- 決策能力
- 知道什麼時候該讓 AI 出手
- 什麼時候該叫它閉嘴的判斷力



架構師在 vibe coding 裡的三大核心任務

不是畫 UML，也不是背名詞。

架構師在 vibe coding 時代要做的，是三件更根本的事情。

這三件事決定了你的系統能不能活下來，能不能擴展，以及最重要的：能不能在兩週後不崩潰。

1 先決定「不做什麼」

Vibe coding 最怕的不是功能不夠，是一開始就做太多。架構師第一個決策是：

- 這個版本要不要 DB？
- 要不要登入？
- 是不是一定要即時？

不做什麼，決定了系統能不能活下來

2 把問題拆成「AI 可以負責的單位」

架構師不是自己下去寫，而是分工：

- 這段邏輯交給 AI agent
- 這段規則要固定，不能讓 AI 自由發揮
- 這段要留給人確認

誰寫的，就誰去除錯。包含 AI。

3 用「決策層」而不是「實作層」思考

Vibe coding 工程師問：要用什麼 API？DB 怎麼設？

架構師先問：這是不是必要的複雜度？這個問題能不能延後？

能延後的，都不是現在該寫的 code



在 vibe coding 時代 你不是先學寫程式，而是先學會當架構師

① 因為只有架構師，才知道什麼時候該讓 AI 出手，什麼時候該叫它閉嘴

疊加式開發的兼容性策略

怎麼一邊做極簡版，一邊又不要把未來鎖死？

答案不是「先把未來都想好」，而是做三件事：

- 留接口
- 留邊界
- 留替換位

這是疊加式開發的核心原則，也是讓你的系統能夠優雅演進的關鍵。



策略一：留接口



先把輸入 / 輸出釘死

一開始就定義兩件事：

- 輸入長什麼樣（欄位、格式、必填）
- 輸出長什麼樣（欄位、格式、狀態）



內部都可以換

中間怎麼算、怎麼存、怎麼呼叫 AI，都可以換。但只要 I/O 固定，你後面加功能會輕鬆很多。



實務一句話

先規格化 I/O，內部先允許很醜

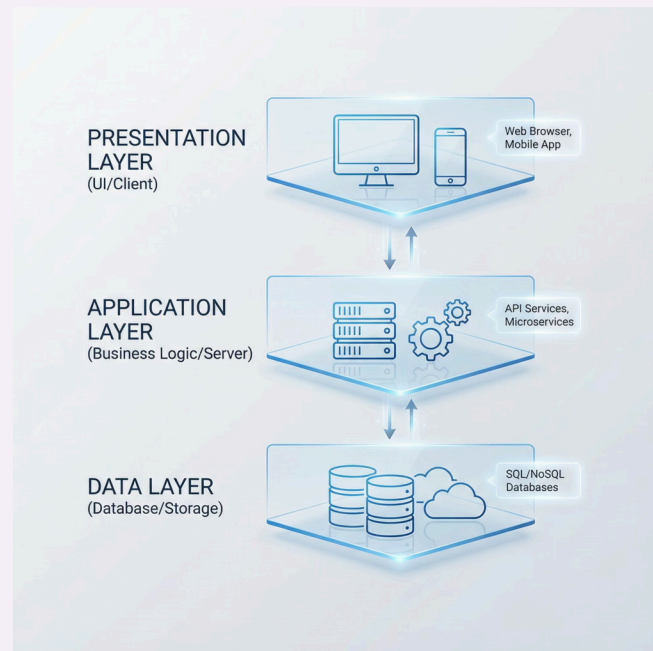
策略二：留邊界

把系統拆成模組，但先不要拆成多服務

你不用一開始就微服務、DDD 全套。你要的是「可替換」和「不互相污染」。最實用的切法通常是三層：

- UI / 前端層：只負責顯示與收集資料，不做商業邏輯
- Domain / 規則層：所有規則都集中在這裡（最值錢的）
- IO / 連接層：DB、第三方 API、AI 模型呼叫都放這裡

規則集中，外部接法隨便換



策略三：留替換位

每個會「變」的點，都先做成可替換選項。你在課程裡最常會遇到的變動點：

資料庫

會從 Notion/Sheet 變成真正資料庫。

AI 模型

會換（或加上 RAG）。

身份驗證

會從無登入變成登入。

權限控制

會從單人變成多人。

流程狀態

會從單步驟變成多狀態。

做法不是先做完，而是先留「切換開關」。先預留選項位置，但不要先做選項內容。

替換位的實務做法

- 先定義 **Storage 介面**

今天是 Sheet，明天是 Postgres，但介面不變

- 先定義 **AIProvider**

今天 GPT，明天換別的，呼叫方式統一

- 先定義 **AuthMode**

none / basic / oauth，可以隨時切換

- 先定義 **Status**

哪怕第一版永遠只有 one status，也先有欄位



最重要的一條 資料結構先保守

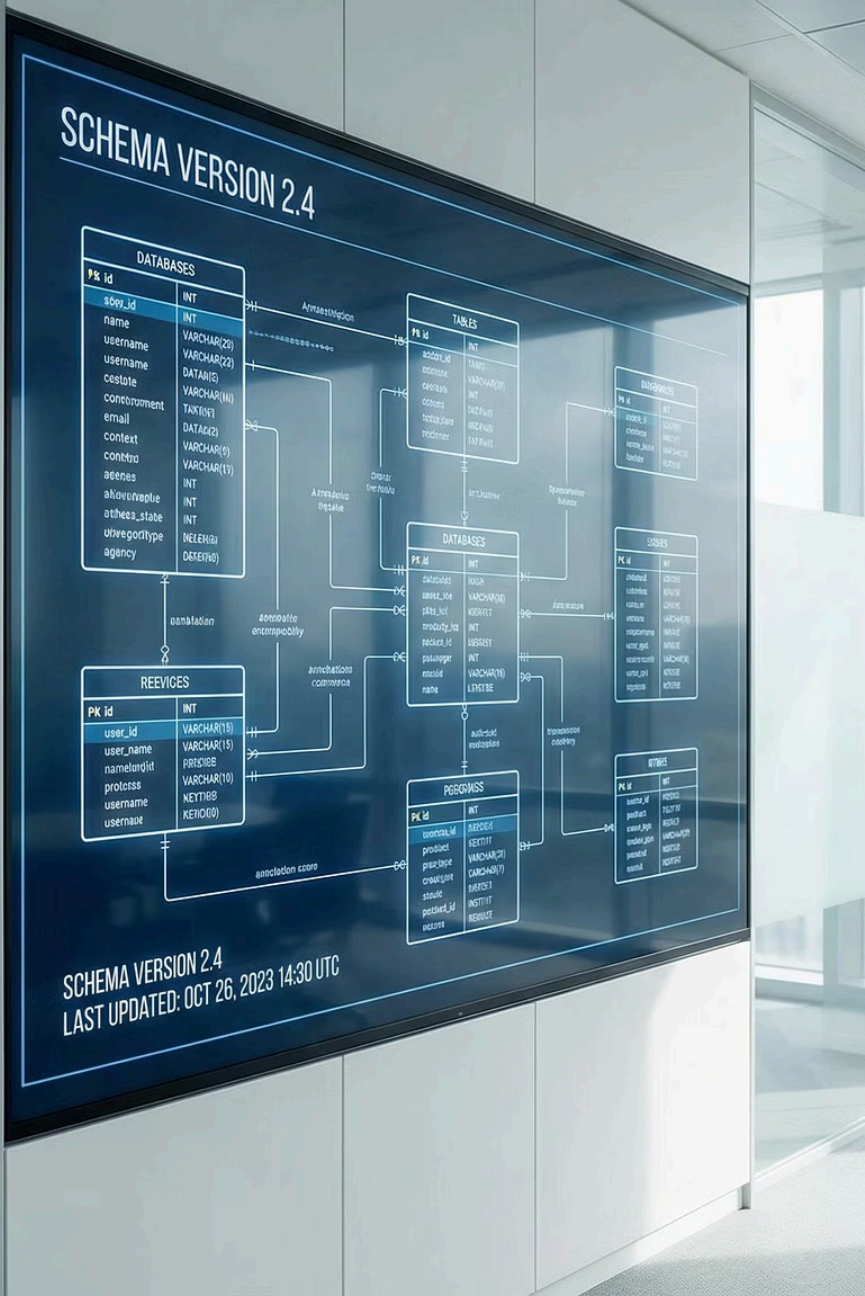
兼容性最常死在兩件事：

- 欄位亂長（後面要改超痛）
- 資料沒有版本（你不知道舊資料怎麼解）

超務實的原則：

- 每筆資料都有 created_at / updated_at
- 每筆資料都有 schema_version

❗ 哪怕你永遠用不到，這兩個欄位會救你命。



疊加式功能的兼容性 Checklist

你每次要加功能前，問自己這 6 句話。只要這 6 題你答得出來，你的系統就很難爆：

1 I/O 影響檢查

這功能會改到現有 I/O 嗎？改了會破壞舊資料嗎？

2 模組邊界檢查

這功能是新模組，還是硬塞進舊模組？

3 替換性檢查

這功能有沒有「可替換位」？未來會不會想換供應商/工具？

4 狀態管理檢查

這功能要不要狀態？狀態從哪來、怎麼轉？

5 失敗情境檢查

這功能的失敗情境是什麼？要怎麼回復？

6 可移除性檢查

如果兩週後要砍掉它，能不能乾淨拔除？

疊加式開發的核心信念

疊加式開發不是「先做少一點」

- 先把接口釘死
- 把規則集中
- 把可變點留成替換位

❗ 你不是在為今天寫 code，你是在幫未來的自己留退路。



W1 架構師最小規格模板

一頁就能填完，填完你就知道現在該做什麼、哪些先不要做、以及未來要加功能不會死。

核心問題定義

這個系統要解什麼問題？一句話說清楚

I/O 規格

輸入什麼？輸出什麼？格式是什麼？

本版本不做清單

列出 3-5 個「這次絕對不做」的功能

三層邊界

UI 層、Domain 層、IO 層各負責什麼？

替換位清單

哪些地方未來會變？預留什麼介面？

資料結構核心欄位

created_at、updated_at、schema_version 都有了嗎？

架構師思維的三個層次



知道不做什麼

最基礎的架構能力



知道怎麼拆

能夠分解問題成可管理單位



知道怎麼留退路

為未來變化預留彈性空間



知道何時該變

最高階的架構判斷力

從VB工程師到架構師的轉變

這不是職稱的改變，而是思維方式的根本轉換。

- 當你開始問「為什麼要做」而不是「怎麼做」
- 當你開始關心系統邊界而不是程式碼細節
- 當你開始為未來留退路而不是急著實作當下

你就已經在用架構師的方式思考了。

1 下一個專案開始前

先填寫「W1 架構師最小規格模板」，
強迫自己想清楚再動手

2 每次要加功能時

先跑一遍「兼容性 Checklist」6 個
問題，確保不會埋地雷

3 每週回顧時

檢查自己問的是「怎麼做」還是「為
什麼做」，調整思維方向

4 每個月反思時

評估自己在三層次（不做什麼、怎麼
拆、怎麼留退路）各處於什麼階段

❗ 在 vibe coding 時代，這個轉變不是選項，是必須。



最後，記住這句話

建立架構師思維 才知道什麼時候該讓 AI 出手 什麼時候該叫它閉嘴

這不是一門技術課程，這是一場思維革命。當你掌握了架構師的思維方式，你就不再是 AI 的使用者，而是 AI 的指揮者。你的價值不在於寫了多少 code，而在於做出了多少正確的決策。

歡迎來到 vibe coding 時代，先當架構師，再決定要不要寫程式。